FIT1056 – Introduction to software engineering
Semester 2
PST4

The Project Journey: An Overview **- Problem-solving tasks 4**
We will use the case study of a Music School Management System (MSMS) to introduce the fundamental software engineering (SE) concepts from the perspective of a developer. Before you begin, review the **Applied#1** slides to make sure you understand how to work with the Git repository.

Music School Management System (MSMS)
(Individual Task)

Objective: You will build a Music School Management System (MSMS) over five stages. Each stage is a direct upgrade of the previous one, taking you from a simple script to a robust, professional application.

Important Notes:

- This is an **individual task**.
- You are **not** required to create a formal Software Requirements Specification (SRS) document.
- You **must** use Git to track your progress. You will make a new "commit" after completing each part.

- **PST1: The Foundation.** Build a simple, in-memory prototype.
- **PST2: The Upgrade.** Add file storage, data validation, and better organization.
- **PST3: The Architecture.** Rebuild with a professional Object-Oriented (OOP) design.
- **PST4: The User Interface.** Replace the text console with a modern Graphical User Interface (GUI).
- **PST5: The Quality Assurance.** Make the application "crash-proof" and prove it works with automated tests.

Part 0: Project Setup and Git Initialization

The same Git steps of PST1 are mentioned here. You may make the changes accordingly as you want.

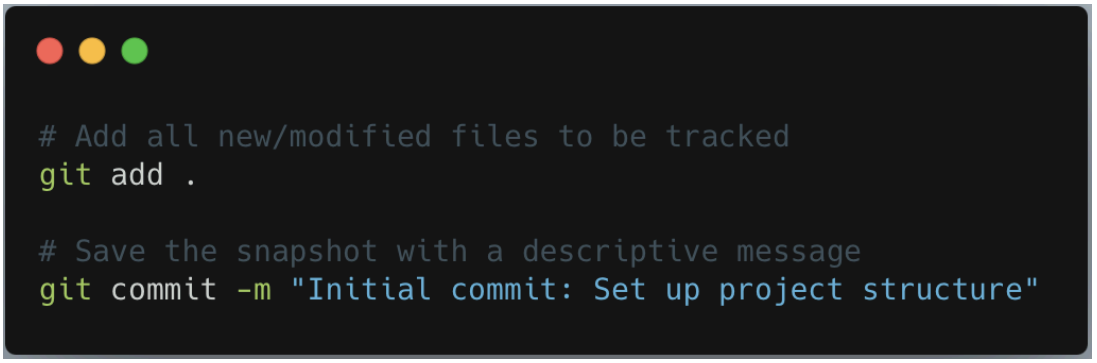
Goal: Create a project directory and initialize it as a Git repository. This is the foundation for your entire project.

Make sure you are on the correct branch (individual). If you have any un-committed changes, commit them now and push. Feel free to push after each individual commit, there is no problem with doing so.

Your Task:

1. Open your terminal or command prompt (Follow the Applied-Week01 Slides).
2. Create a new folder for your project (e.g., msms-project) and navigate into it.
3. Initialize a new Git repository in this folder. This command creates a hidden .git subdirectory that will track all your changes.

4. Create your main source code file. (e.g., main.py).
5. Add this new file to the Git staging area and make your first commit. A commit is a snapshot of your code at a specific point in time.



Part 4 (PST4): The Modern User Experience (GUI)

Your Goal: To solve the "Clunky & Unfriendly" problem of the text-based console. You will build a Graphical User Interface (GUI) that allows the receptionist to interact with the system using buttons, forms, and tables. This GUI layer will be completely separate from your application logic, calling methods on the ScheduleManager you perfected in PST3.

Technology Choice: We will use **Streamlit** for this example because it allows us to build beautiful, interactive web-based UIs with pure Python, making it fast and easy to get started.

The New Directory Structure: We add a dedicated gui directory for all our interface code.

```
.
├── app/
│   └── (all your PST3 files: schedule.py, user.py, etc.)
├── data/
│   └── msms.json
├── gui/
│   ├── __init__.py
│   ├── main_dashboard.py    # The main application layout and navigation
│   ├── student_pages.py     # All GUI components for student management
│   └── roster_pages.py      # All GUI components for scheduling and rosters
└── main.py                  # This will now just launch the GUI```
```

Fragment 4.1: The GUI Entry Point & Main Dashboard

Mission: Our first step is to establish the main application window and the primary receptionist dashboard. This dashboard will serve as the central hub, featuring a navigation menu in a sidebar that allows the user to seamlessly switch between different pages, such as "Student Management" and "Daily Roster".

Your Task:

- Begin by installing the Streamlit library via your terminal: pip install streamlit.
- Refactor your top-level main.py file. Its sole responsibility will now be to launch the GUI by importing and calling a function from our new gui package.
- Next, in gui/main_dashboard.py, you will create the launch() function. This function will configure the main page layout, instantiate our ScheduleManager, and leverage Streamlit's session state to ensure the manager object persists as the user navigates between pages. It will also render the main sidebar navigation menu.
- **Check the Templates give in:**
    - **main.py (The New Launcher)**
    - **gui/main_dashboard.py**
- **How to Run:** In your terminal, execute the command: streamlit run main.py.

Checkpoint: Commit Your Progress
Save your file and commit the change.

```
git add gui/main_dashboard.py main.py
git commit -m "feat(gui): Implement main Streamlit dashboard with navigation and session state"
```

(Note: "Feat" is a common convention for a commit that introduces a new feature.)

Fragment 4.2: The Student Management Page

Mission: To build the GUI page for student management, now with a fully functional registration form.

Your Task:

- Create the file gui/student_pages.py.
- Implement the show_student_management_page(manager) function.
- The "Register Student" form will call manager.register_new_student(name, instrument) when the submit button is pressed.
- **Check the Template give in:**
    - **gui/student_pages.py**
- **Update gui/main_dashboard.py**: Remember to import show_student_management_page and call it within the appropriate if block in your launch function.

Checkpoint: Commit Your Progress
Save your file and commit your new function.

```
git add gui/student_pages.py gui/main_dashboard.py
git commit -m "feat(gui): Build student management page with search and registration forms"
```

Fragment 4.3: The Daily Roster & Check-In Page

Mission: We will now build a visual and interactive page for viewing the daily class schedule and performing real-time student check-ins.

Your Task:

- Create the file gui/roster_pages.py.
- Implement the show_roster_page(manager) function.
- The "Check-in" section will call manager.check_in(student_id, course_id) when the submit button is pressed and can provide appropriate success or error feedback to the user.
- **Check the Template give in:**
    - **gui/roster_pages.py**
- **Update gui/main_dashboard.py**: Import show_roster_page and call it within the appropriate elif block in your launch function.

Checkpoint: Commit Your Progress
Save your file and commit this important update.

```
git add gui/
git commit -m "feat(gui): Implement daily roster and check-in pages, completing PST4"
```

```
########################################
# 4  Push the new commit(s) up to GitHub
########################################
git push origin individual
#          |      |
#          |      └─ the branch name on GitHub
#          └──────── the remote repository (GitHub by default)
```

Instructions

- **Template files provided:** You will find a skeleton for every task (Fragment#_#.py)
- **Finish • Run • Test:** Complete each fragment, run it locally, and confirm it works.
- **Commit & push:** Test the complete application, then commit and push.
- **One README to rule them all:** Create **one** high-quality README.md at the root of your repo that explains
    - what each part does,
    - how to run and test the full program, any design choices or assumptions you made.
- **A clear, detailed README is worth marks, treat it like part of the assignment**

Submission Information

- Please ensure you submit your work on Moodle before the deadline to avoid any late penalties.
- Upload your task files as a single ZIP file.
- In addition, make sure to commit and push your code to Git before the same deadline.